

SkipVLA: Skipping VLA Steps with Classical Planning for Fast Robot Manipulation

Kaivalya Agrawal, Md Ashiqur Rahman, Raymond A. Yeh, and Zachary Kingston

Abstract—Vision-Language-Action (VLA) models are a class of generalist robot policies that map camera images and language instructions directly to robot actions. While promising, these models remain slow at test time, particularly for long-horizon tasks that require many queries to the policy. Recent efforts reduce VLA latency by distilling smaller models, overlapping asynchronous action chunks, or pairing the VLA with a fast low-level policy, but still run a learned policy for the entire task. In contrast to VLA, classical motion planners quickly find collision-free motions, but require an explicit goal and have no semantic understanding of the task. In this work, we present SkipVLA, a hybrid policy that combines a pretrained VLA with a classical motion planner, using the planner for free-space motion and querying the VLA only for contact-rich skills such as grasping and placing. SkipVLA reuses the frozen vision-language backbone of the VLA to predict a target pose for each planned motion, and learns this predictor without additional demonstrations introduced into the system by using what was already learnt by the large VLA. We evaluate SkipVLA with three VLAs on 13 LIBERO tasks in simulation and three pick-and-place tasks on a physical 6-DoF YAM arm, demonstrating up to 2.5x faster task completion and significantly lower energy consumption while achieving the same task success rate.

I. INTRODUCTION

Vision-Language-Action (VLA) models [1, 2] have emerged as a promising approach for generalist robot manipulation. By pairing pretrained vision-language backbones with expressive action heads, VLAs map raw camera observations and natural-language task instructions directly to robot actions. This unified formulation enables open-vocabulary instruction following, cross-scene generalization, and long-horizon task execution without handcrafted heuristics.

Despite their expressive capability, VLAs face a severe practical limitation: high inference latency. Generating each action chunk requires forward passes through multi-billion-parameter networks, resulting in latencies that far exceed the robot’s control cycle. In long-horizon tasks requiring dozens or hundreds of queries, this overhead leads to sluggish execution, high energy consumption on edge compute platforms, and compromised reactive safety.

Prior work to reduce VLA latency typically distills smaller models [3], overlaps asynchronous action chunks [4], or pairs the VLA with a fast low-level policy in a hierarchical setup [5]. However, each introduces notable trade-offs: distilled models frequently suffer performance degradation; asynchronous chunking risks executing stale actions that push the policy out of distribution; and hierarchical setups require

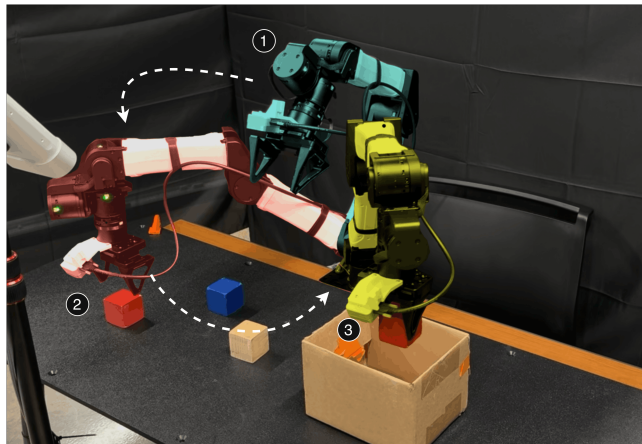


Fig. 1: Our hybrid method on a real-world pick-and-place task, shown at three points in time. White arrows are free-space transit executed by the planner module. (❶, cyan) From the start pose, the planning module predicts a hover target above the red block and moves the arm to it. (❷, red) The VLA expert module performs the grasp. When the gripper closes, control returns to the planner, which moves the arm to a predicted hover target above the box. (❸, yellow) The VLA expert places the block in the box.

training a completely new policy network from scratch for specialized subtasks. Crucially, all of these methods treat manipulation as a monolithic end-to-end learning problem, querying large neural networks even during simple, unconstrained motions.

In reality, manipulation tasks exhibit a natural physical decomposition: they alternate between short, *contact-rich* manipulation skills (e.g., grasping, reorienting, or placing) and long stretches of *free-space transit* (e.g., reaching toward an object or navigating to a receptacle). Contact-rich manipulation requires adaptive, closed-loop visuomotor feedback to manage complex physical contacts, a domain where learned VLAs excel. In contrast, free-space transit is a purely geometric and kinematic problem that requires collision avoidance but minimal semantic reasoning.

Classical motion planners, such as sampling-based algorithms [6] and trajectory optimizers [7], solve this geometric problem rapidly, generating collision-free trajectories in microseconds to milliseconds with formal safety guarantees. However, classical planners are semantically blind, i.e., they require explicit goal poses and cannot infer them from natural language. Methods that prompt foundation models for zero-shot spatial keypoints [8–10] and only rely on geometric optimization for the entire task struggle with the delicate contact dynamics that VLAs naturally handle.

This motivates a hybrid architecture that reserves the VLA for contact-rich interactions, offloads free-space transit to a

fast classical planner, and uses the VLA’s frozen representations to predict the planner’s target poses.

To this end, we introduce SkipVLA, a modular hybrid policy that dynamically interleaves a pretrained VLA with a classical motion planner. SkipVLA scores candidate 3D object bounding boxes, conditioned on the base VLA’s frozen vision-language features, to determine collision-free hover targets. A classical planner navigates the robot through free space to the target pose, then hands control to the VLA for contact-rich manipulation. Once the manipulation completes as signaled by a gripper state transition, control returns to the planner. We train the target predictor using self-supervision extracted from existing demonstration trajectories at gripper events, requiring no manual annotations, architectural changes, or policy retraining. **Our main contributions are:**

- We present SkipVLA, a plug-and-play framework that accelerates manipulation by routing free-space transit to classical motion planners and reserving pretrained VLAs solely for contact-rich skills.
- We develop a lightweight scoring head that uses the VLA’s frozen vision-language tokens to predict 3D target poses and can be trained on an existing imitation learning dataset with no additional annotation.
- We evaluate SkipVLA across 13 tasks on the LIBERO benchmark in simulation and three physical tasks on a 6-DoF YAM arm, spanning three distinct VLA backbones ($\pi_{0.5}$, SmoVLA, and MolmoAct2) and multiple motion planners (VAMP and cuRoboV2).
- SkipVLA achieves up to a $2.5\times$ wall-clock speedup (59.6% latency reduction) and cuts compute energy by over 52% on an NVIDIA Jetson Thor.

II. RELATED WORK

Vision-language-action models. Following the success of Vision-Language Models (VLMs) [11, 12] in scene understanding and cross-modal semantic alignment, substantial research has aimed to construct open-ended robotic policies that unify perception and control within a single end-to-end framework, known as VLA models. Recently, VLAs have expanded to incorporate generative architectures for better generalization and performance, such as diffusion- and flow-matching-based action heads [2, 13, 14], to capture complex, multimodal action distributions. While these models have been successful in executing semantically aligned, long-horizon tasks, their practical deployment is severely bottlenecked due to their massive parameter counts, making them unsuitable for high-frequency real-world control.

Efficient VLA architectures. Extensive research has been conducted to reduce the high inference latency of VLAs. Large VLAs have been distilled into smaller variants to reduce parameter counts and improve inference speed [3, 15]. However, such models often exhibit drops in performance due to distribution shift and still remain relatively slow for high-frequency control. Other works employ action chunking [4, 16, 17] to overlap asynchronously inferred action chunks, effectively hiding execution pauses due to slow inference. These approaches introduce staleness in the inferred chunk,

and interleaving action chunks can push the policy off distribution. Finally, hierarchical methods introduce a low-latency, smaller policy module that operates asynchronously alongside a large VLA [5, 18–21], reusing the visual representations of the larger network.

Classical motion planning. Classical planning approaches like Sampling-Based Motion Planning algorithms (SBMP) and trajectory optimization methods have long proven to be successful in solving complex navigation and manipulation tasks. Modern implementations of SBMPs are highly energy- and time-efficient, capable of generating collision-free trajectories in microseconds [6, 22]. Similarly, gradient-based trajectory optimization frameworks [23, 24] and GPU-accelerated solvers [7] calculate dynamically feasible motion in real time. Despite their safety guarantees and low-latency planning, classical planners are semantically blind. They operate purely on coordinate geometry and require well-defined goal states or objective functions.

Combining learning with classical planning. Recent works have attempted to combine the semantic reasoning of foundation models with the reliability of classical planners. One line of research, including VoxPoser [8] and ReKep [9], prompts VLMs to output spatial cost maps or relational constraints that are then solved by trajectory optimization. However, because these methods rely entirely on zero-shot geometric optimization rather than learned action priors, they struggle to execute dynamic, contact-rich manipulation that cannot be represented as a cost function.

A parallel line of work attempts to integrate classical planning directly with trained behavioral policies like LiLoVLA [25] and MPVI [26]. However, these methods suffer from distinct architectural bottlenecks. Frameworks like MPVI treat the motion planner primarily as a recovery mechanism, using it to veer the policy back to in-distribution only after it drifts out-of-distribution. Other works, like LiLoVLA, rely on pre-defined action sequences before performing simple skill chaining.

In contrast to these frameworks, SkipVLA interleaves a generalist VLA with classical sampling-based motion planning at planning and execution time. By leveraging the VLA’s pretrained representations strictly for target pose prediction, we dynamically hand off all free-space navigation in between atomic skills to the fast classical planner. This achieves massive latency reductions and guarantees collision-free transit without compromising on task success rate or introducing shifts in policy and data distribution within the VLA and builds on top of the base VLA architecture.

III. BACKGROUND

In this section, we introduce the background and notation used in this work.

VLA. We denote a generalist VLA robot policy as π^{vla} . This policy maps an observation $\mathcal{O}_s$, a natural-language instruction L (task description), and an optional proprioceptive state $\mathbf{p}_s$ to a robot action

$$a_s = \pi^{\text{vla}}(\mathcal{O}_s, L, \mathbf{p}_s) \quad (1)$$

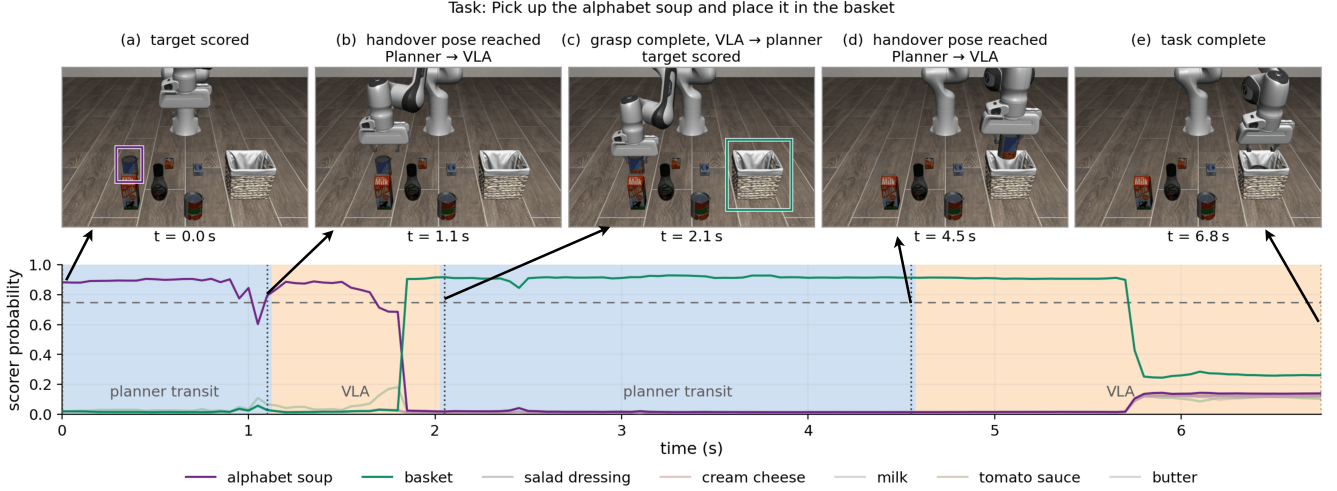


Fig. 2: **SkipVLA on a single LIBERO-Object episode.** Top: five keyframes from one successful rollout of $\pi_{0.5}$ + SkipVLA with the VAMP planner. (a) The scoring module selects a target object and pose. (b) VAMP’s plan is followed from start to target pose and control is handed over to VLA. (c) The VLA completes the skill of grasping and the control is handed over to SkipVLA which scores the next target. (d) VAMP’s plan is followed from start to target pose and control is handed over to VLA. (e) The VLA completes the skill of placing and task is marked complete. Bottom: the scoring module’s probability over all seven tracked object bounding boxes for the same episode, with background shading showing which policy holds control at each step: planner transit (blue) and VLA (orange). Control returns to the planner at exactly the grasp and release transitions. Over the 6.8s episode, the planner drives 54% of the task. The deployed system queried the scorer twice; the continuous probability curves are an offline re-scoring of every frame with a fresh latent to show the change in the scoring signal throughout the trajectory.

at step s . The robot’s proprioceptive state $\mathbf{p}_s$ consists of the joint angles and gripper state (with corresponding end-effector pose $\mathbf{p}_s^{ee}$). The observation $\mathcal{O}_s$ includes an image and depth map from a workspace viewpoint RGBD camera and an image from a wrist-mounted RGB camera.

Trajectory. We use the term *trajectory* to denote a sequence of proprioceptive states τ as

$$\tau = (\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_N), \quad (2)$$

where $N = |\tau|$ is the trajectory length and $a_s = \mathbf{p}_{s+1} - \mathbf{p}_s$ is the action taken at step s that changes the robot’s proprioceptive state from $\mathbf{p}_s$ to $\mathbf{p}_{s+1}$.

Action chunks. Different VLA architectures generate actions at different granularities. A VLA when queried at step s can produce an action *chunk* of K future actions

$$A_s = (a_s, a_{s+1}, \dots, a_{s+K-1}) = \pi^{\text{vla}}(\mathcal{O}_s, L, \mathbf{p}_s). \quad (3)$$

For a given VLA, the action chunk horizon $K \geq 1$ is a fixed constant. Therefore, the VLA is queried every K steps. We denote the set of these *query steps* by $\mathcal{Q} = \{1, 1+K, 1+2K, \dots\} \subset \{1, \dots, N\}$, with $|\mathcal{Q}| = \lceil N/K \rceil$.

Execution time. The total execution time for a trajectory τ of length $N = |\tau|$ is governed by two factors: the control period $\Delta t_{\text{ctrl}} = \frac{1}{f_{\text{ctrl}}}$, dictated by the sampling frequency f_{ctrl} of the robot controller, and the inference latency Δt_{lat} , which is the processing time required by the VLA to generate each action chunk.

The total execution time of the trajectory τ generated by policy π is given by:

$$\mathcal{T}(\tau) = N \cdot \Delta t_{\text{ctrl}} + \sum_{s \in \mathcal{Q}} \Delta t_{\text{lat}} = N \left(\Delta t_{\text{ctrl}} + \frac{\Delta t_{\text{lat}}}{K} \right), \quad (4)$$

where Δt_{ctrl} and Δt_{lat} are the control period and inference latency of π , and $|\mathcal{Q}| = N/K$ is the number of query steps (assuming N is a multiple of K for simplicity). For a generalist VLA with a large number of parameters, Δt_{lat} is often the bottleneck of the execution time ($\Delta t_{\text{lat}} \gg \Delta t_{\text{ctrl}}$).

Imitation learning. In robot manipulation, policies are mainly trained via imitation learning on a static dataset $\mathcal{D} = \{(\tau^{e,i}, \{\mathcal{O}\}^i, L^i)\}$ of expert demonstrations. The objective trains the policy to imitate expert actions step-by-step:

$$\min_{\pi} \mathbb{E}_{(\tau^e, \{\mathcal{O}\}, L) \sim \mathcal{D}} \sum_{s=1}^{|\tau^e|} \ell(a_s^e, \pi(\mathcal{O}_s, L, \mathbf{p}_s)), \quad (5)$$

where a_s^e is the expert action at step s and ℓ is an appropriate loss for the task.

IV. METHOD

We propose SkipVLA which augments a generalist VLA policy π^{vla} with a lightweight, low-latency classical control module π^{fast} ($\Delta t_{\text{lat}}^{\text{fast}} \ll \Delta t_{\text{lat}}^{\text{vla}}$). It dynamically routes between classical motion planning (for free-space transit) and a VLA (for contact-rich manipulation) (see Fig. 2).

A. Problem Formulation and Objective

We achieve this hybrid execution through an indicator module I that determines which policy to execute at each timestep. Given an observation $\mathcal{O}_s$, task instruction L , and proprioceptive state $\mathbf{p}_s$, the indicator outputs a binary decision $I_s \in \{0, 1\}$, yielding the hybrid policy:

$$\pi^{\text{hyb}}(\mathcal{O}_s, L, \mathbf{p}_s) \triangleq \begin{cases} \pi^{\text{vla}}(\mathcal{O}_s, L, \mathbf{p}_s), & I_s = 0, \\ \pi^{\text{fast}}(\mathcal{O}_s, L, \mathbf{p}_s), & \text{otherwise.} \end{cases} \quad (6)$$

The latency of the hybrid policy at step s is expressed as:

$$\Delta t_s^{\text{lat}} = \begin{cases} \Delta t_{\text{lat}}^{\text{vla}}, & I_s = 0, \\ \Delta t_{\text{lat}}^{\text{fast}}, & \text{otherwise.} \end{cases} \quad (7)$$

Trajectory-level objective. To keep the latency low, the π^{fast} module is designed to be lightweight using classical motion planning. However, classical motion planners (such as RRT or PRM) generate collision-free paths that optimize geometric or kinematic criteria rather than mimicking human demonstration trajectories step by step. Consequently, a per-action imitation loss as in Eq. (5) would incorrectly penalize such valid transit trajectories, even when they reach the destination or the target pose safely and faster.

To address this, we use a trajectory-level *imitation learning* objective where, rather than matching every action, the trajectory rolled out by π^{hyb} is constrained to pass through key *waypoints* of the expert trajectory, which ensures successful task completion. To minimize total execution time, the overall objective can be expressed as follows:

$$\min_{\pi^{\text{fast}}, I} \mathbb{E}_{\tau^e \sim \mathcal{D}} \left[\mathcal{T}(\tau^{\text{hyb}}) + \lambda \cdot \ell_{\text{traj}}(\tau^{\text{hyb}}, \tau^e) \right], \quad (8)$$

where τ^{hyb} is the trajectory rolled out by π^{hyb} , τ^e is the corresponding expert trajectory, $\mathcal{T}(\tau^{\text{hyb}})$ is the execution time, and $\lambda > 0$ is a hyper-parameter of choice. $\ell_{\text{traj}}(\tau^{\text{hyb}}, \tau^e) = 0$ implies the hybrid trajectory passes through all required key waypoints, ensuring task completion.

Key-waypoints extraction. The trajectory τ for completing a task can be partitioned into segments, each associated with a subtask. Broadly, these subtasks fall into two categories: *active object manipulation*, where the robot’s end-effector or gripper performs grasping, reorientation, or other contact-rich interactions with the object; and *reach-and-place* (approach/placement), where the robot primarily moves its end-effector with or without a grasped object.

In more detail, given a trajectory τ , we partition it as $\tau = [\tau^1, \tau^2 \dots \tau^n]$, where we concatenate each segment τ^j corresponding to either active object manipulation or reach-and-place.

For an expert trajectory segment $\tau^{e,j}$ corresponding to active object manipulation, the matching segment $\tau^{\text{hyb},j}$ rolled out by π^{hyb} should closely follow the expert at each step to ensure successful completion of the subgoal. On the other hand, for an expert reach-and-place segment $\tau^{e,j}$, successful completion requires the corresponding segment $\tau^{\text{hyb},j}$ to match only the initial and final end-effector poses. Based on this structure, we define the segment-level loss as follows:

$$\ell_{\text{traj}}(\tau^{\text{hyb},j}, \tau^{e,j}) = \begin{cases} \sum_{s=1}^{N_j^e} d(\tau_s^{\text{hyb},j}, \tau_s^{e,j})^2, & \text{if manipulation,} \\ d(\tau_1^{\text{hyb},j}, \tau_1^{e,j})^2 + d(\tau_{N_j^e}^{\text{hyb},j}, \tau_{N_j^e}^{e,j})^2, & \text{if reach-and-place,} \end{cases} \quad (9)$$

where $\tau_s^{\text{hyb},j}$ and $\tau_s^{e,j}$ represent the end-effector poses at step s for the respective segments. Additionally, $N_j = |\tau^{\text{hyb},j}|$ and $N_j^e = |\tau^{e,j}|$ indicate the lengths of the respective segments, while $d(\cdot, \cdot)$ denotes the chosen distance metric.

The full trajectory-level loss is then defined as the sum of segment-level losses as:

$$\ell_{\text{traj}}(\tau^{\text{hyb}}, \tau^e) = \sum_{j=1}^n \ell_{\text{traj}}(\tau^{\text{hyb},j}, \tau^{e,j}). \quad (10)$$

B. Implementation Details

In this section, we detail the design of the fast control module π^{fast} and the policy indicator module I . Our hybrid policy π^{hyb} is designed to allocate tasks to the policies based on their complexity: π^{fast} executes reach-and-place subtasks, while π^{vla} handles active object manipulation. This division leverages the fact that contact-rich manipulation demands expressive, learned visuomotor control, whereas free-space transit is purely geometric and can be completed significantly faster using classical motion planning.

Fast control module π^{fast} . The fast control module π^{fast} is composed of two key components: a *target pose predictor* f_{tgt} and a *low-level motion planner* $\mathcal{P}$.

The target pose predictor f_{tgt} maps the current observation $\mathcal{O}_s$, language instruction L , and current proprioceptive state $\mathbf{p}_s$ to the target end-effector pose at the end of the current subtask (trajectory segment), denoted by $\mathbf{p}_s^{\text{tgt}}$ as:

$$\mathbf{p}_s^{\text{tgt}} = f_{\text{tgt}}(\mathcal{O}_s, L, \mathbf{p}_s). \quad (11)$$

The low-level motion planner $\mathcal{P}$ then takes the current state $\mathbf{p}_s$, target pose $\mathbf{p}_s^{\text{tgt}}$, and current observation $\mathcal{O}_s$ to generate an action sequence $(a_s, \dots, a_{s+m})$ of horizon m that moves the robot from $\mathbf{p}_s$ to $\mathbf{p}_s^{\text{tgt}}$ while avoiding collisions, i.e.,

$$(a_s, \dots, a_{s+m}) = \mathcal{P}(\mathbf{p}_s, \mathbf{p}_s^{\text{tgt}}, \mathcal{O}_s). \quad (12)$$

Target pose predictor f_{tgt} . The target pose predictor identifies the relevant object for the current subtask and determines a collision-free hover pose directly above it. Specifically, from the observation $\mathcal{O}_s$, we first extract 3D bounding boxes $\mathcal{B} = \{\mathbf{b}_i\}_{i=1}^M$ for the objects in the scene, where $\mathbf{b}_i = (\mathbf{c}_i, \mathbf{e}_i)$ consists of 3D centroid $\mathbf{c}_i \in \mathbb{R}^3$ and 3D extents $\mathbf{e}_i \in \mathbb{R}^3$.

We score candidate object boxes by cross-attention between a learned positional embedding of each bounding box $\mathbf{b}_i$ (acting as the query) and the vision-language feature tokens extracted from the scene (acting as keys and values). More concretely, for the i -th object bounding box $\mathbf{b}_i$, its relevance score r_i is computed as:

$$r_i = \text{MLP} \left(\text{Softmax} \left(\frac{\mathbf{q}_i^{\text{box}} \mathbf{K}_{\text{vl}}^\top}{\sqrt{d_k}} \right) \mathbf{V}_{\text{vl}} \right), \quad (13)$$

where $\mathbf{q}_i^{\text{box}} = \phi_{\text{pos}}(\mathbf{b}_i) \in \mathbb{R}^{d_k}$ is the learned positional embedding of the box, $\mathbf{K}_{\text{vl}}, \mathbf{V}_{\text{vl}} \in \mathbb{R}^{T \times d_k}$ are linear projections of the T vision-language feature tokens $\mathbf{F}_{\text{vl}}$. A lightweight MLP head then maps the attended feature representation to a scalar score $r_i \in \mathbb{R}$. The target object is selected via $\hat{i} = \arg \max_i r_i$, and a top-down, collision-free hover pose above candidate box $\mathbf{b}_{\hat{i}}$ is assigned as the target pose $\mathbf{p}_s^{\text{tgt}}$.

We extract the vision-language features $\mathbf{F}_{\text{vl}}$ using the vision-language backbone of π^{vla} to reuse its pretrained

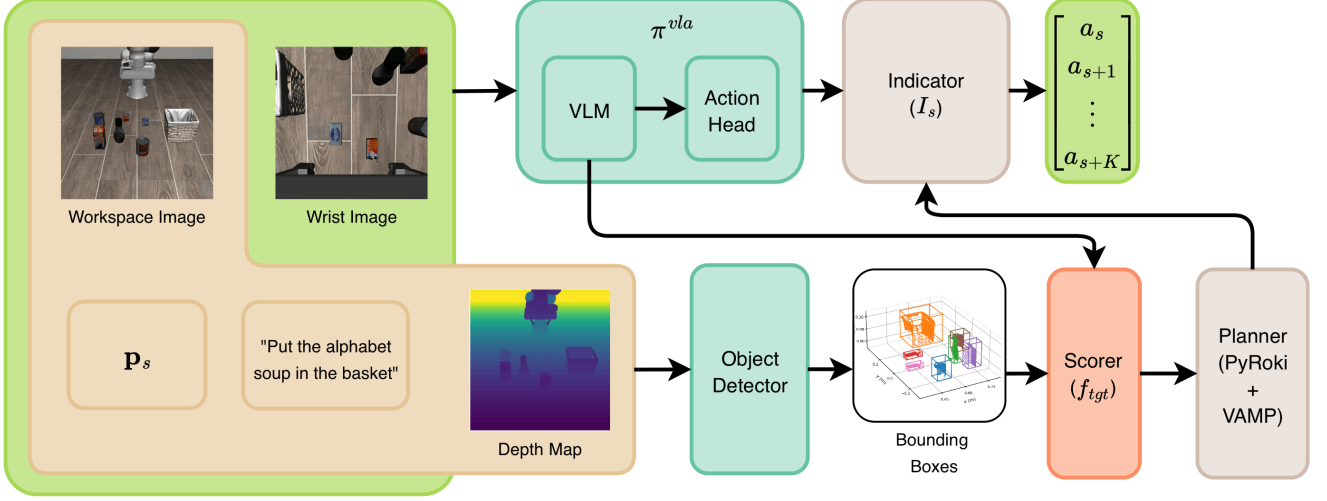


Fig. 3: **Inference pipeline of our proposed SkipVLA.** We represent our input data as $\mathcal{O}_s, L, \mathbf{p}_s$, where $\mathcal{O}_s$ contains a workspace image, a wrist image, and a workspace depth map, L represents the language instruction of the task, and $\mathbf{p}_s$ represents the proprioceptive state. The base expert component π^{vla} comprises a VLM and an action head that process the input stream to output an action vector and a vision-language context latent. The object detection module takes in a different subset of inputs and uses FastSAM [27] and DBSCAN to generate object bounding boxes. The scoring module f_{tgt} selects the target object given the bounding boxes and latent context from the VLM. We use PyRoki [28] and VAMP to generate an action sequence. Finally, the indicator module I decides which action sequence to execute.

representations. Only the positional embedding ϕ_{pos} , cross-attention projections, and the scoring MLP are trained from scratch. The target pose predictor is only called once at the beginning of every reach-and-place phase ($I = 1$).

Low-level motion planner $\mathcal{P}$. Our framework is agnostic to the planner choice, and we instantiate $\mathcal{P}$ using classical motion planners such as cuRoboV2 and VAMP.

Policy indicator module I . Learning task boundaries directly from visual observations is an active research area. As in this work, we focus on hybridizing the VLA with a fast motion planner; we choose an effective and fast heuristic based on gripper state and target pose proximity to indicate the task boundaries.

The policy begins in a reach-and-place phase ($I_0 = 1$), using π^{fast} to navigate toward the target pose $\mathbf{p}_s^{tgt}$. Once the end-effector reaches the target ($d(\mathbf{p}_s^{ee}, \mathbf{p}_s^{tgt}) < \epsilon_d$, where $\epsilon_d > 0$ is some tolerance threshold), control is handed over to π^{vla} ($I_s = 0$). Once active manipulation is complete, signaled by a toggle in the gripper state ($g_s \neq g_{s-1}$), the indicator switches back to $I_s = 1$, returning control to π^{fast} . Formally, let $g_s \in \{0, 1\}$ be the binary gripper state (open or closed) at step s , $\mathbf{p}_s^{ee}$ be the end-effector pose, and $\mathbf{p}_s^{tgt}$ be the active target pose. With initial state $I_0 = 1$, the indicator I_s is recursively defined as:

$$I_s = \begin{cases} 1, & \text{if } g_s \neq g_{s-1}, \\ 0, & \text{if } I_{s-1} = 1 \text{ and } d(\mathbf{p}_s^{ee}, \mathbf{p}_s^{tgt}) < \epsilon_d, \\ I_{s-1}, & \text{otherwise.} \end{cases} \quad (14)$$

C. Training the Hybrid Architecture

Our hybrid policy design naturally simplifies the optimization in Eq. (8) and does not require end-to-end policy

optimization. First, we freeze the pretrained π^{vla} , which already handles contact-rich manipulation. Furthermore, $\mathcal{P}$ guarantees reachability to any valid goal pose with minimal latency; the reach-and-place trajectory loss in Eq. (9) is minimized whenever the target pose $\mathbf{p}_s^{tgt}$ is correctly predicted. Consequently, optimizing Eq. (8) reduces solely to training the target pose predictor f_{tgt} (specifically its box-scoring head) to identify the target object.

Automated data generation and training. To train the target scoring function without manual annotation, we extract supervision directly from expert demonstration trajectories. Reach-and-place segments correspond to free-space transit motions that precede an interaction. We detect the completion of each reach-and-place segment by identifying gripper events, namely any change in the gripper state across a threshold (closing to grasp or opening to release). At this gripper event, the target object or receptacle is identified by finding the bounding box closest to the robot’s end-effector $\mathbf{p}_{event}^{ee}$ as:

$$i^* = \arg \min_i \|\mathbf{p}_{event}^{ee} - \mathbf{c}_i\|_2. \quad (15)$$

We then assign i^* as the ground-truth target for all preceding states $\mathcal{O}_s$ within that reach-and-place segment. For any observation $\mathcal{O}_s$, given candidate bounding boxes $\mathcal{B}$ and frozen vision-language tokens $\mathbf{F}_{vl}$, the scoring head computes relevance scores $\{r_i\}_{i=1}^M$ according to Eq. (13). We optimize the trainable parameters of f_{tgt} using a cross-entropy loss:

$$\mathcal{L}_{tgt} = -\log \frac{\exp(r_{i^*})}{\sum_{j=1}^M \exp(r_j)}. \quad (16)$$

This allows SkipVLA to learn robust target prediction across varying viewpoints throughout transit, requiring no additional

Policy	SR (%) $\uparrow$	n_{succ}	Wall Clock Time (s) $\downarrow$	VLA q $\downarrow$
$\pi_{0.5}$ (base)	99.0	495	8.33 $\pm$ 1.18 (8.3)	17.0 $\pm$ 2.4 (17)
+ SkipVLA + VAMP	99.0	495	<u>5.27$\pm$2.19 (4.4)</u>	<u>10.6$\pm$4.7 (9)</u>
+ SkipVLA + cuRoboV2	<u>99.2</u>	<u>496</u>	6.28 $\pm$ 2.82 (5.3)	9.9$\pm$4.8 (8)
+ Heuristic Ablation	99.6	498	5.11$\pm$1.16 (4.7)	10.7 $\pm$ 2.2 (10)
SmolVLA (base)	39.0	195	12.61 $\pm$ 4.99 (10.8)	21.3 $\pm$ 8.3 (18)
+ SkipVLA + VAMP	77.0	385	6.96$\pm$3.76 (5.8)	<u>11.4$\pm$6.2 (10)</u>
+ SkipVLA + cuRoboV2	82.0	410	7.85 $\pm$ 3.94 (6.8)	10.8$\pm$6.5 (9)
+ Heuristic Ablation	<u>79.0</u>	<u>395</u>	<u>7.25$\pm$3.02 (6.2)</u>	12.6 $\pm$ 5.0 (11)

TABLE I: **LIBERO-object Evaluation.** We evaluate SkipVLA across 10 pick and place tasks in simulation. We also ablate the indicator heuristics to show the model’s adaptability. We calculate statistics over successful trials only. **Bold:** best; underline: second-best in each backbone block. Numbers shown as Mean $\pm$ Standard Deviation (Median).

Policy	SR (%) $\uparrow$	n_{succ}	Wall Clock Time (s) $\downarrow$	VLA q $\downarrow$
$\pi_{0.5}$ (base)	<u>94.7</u>	<u>142</u>	15.69 $\pm$ 2.35 (15.2)	31.7 $\pm$ 4.7 (31)
+ SkipVLA + VAMP	<u>94.7</u>	<u>142</u>	13.84$\pm$7.07 (11.0)	<u>26.5$\pm$14.4 (21)</u>
+ SkipVLA + cuRoboV2	95.3	143	17.84 $\pm$ 4.99 (16.3)	24.1$\pm$9.9 (21)
+ Heuristic Ablation	<u>94.7</u>	<u>142</u>	<u>15.28$\pm$6.02 (13.2)</u>	31.4 $\pm$ 12.0 (27)
SmolVLA (base)	5.3	8	26.91 $\pm$ 7.40 (23.7)	45.0 $\pm$ 12.3 (39.5)
+ SkipVLA + VAMP	24.7	37	15.29$\pm$9.17 (11.3)	<u>24.6$\pm$15.8 (17)</u>
+ SkipVLA + cuRoboV2	29.3	44	17.97 $\pm$ 8.02 (15.1)	22.8$\pm$12.4 (18.5)
+ Heuristic Ablation	<u>26.7</u>	<u>40</u>	<u>15.61$\pm$4.75 (14.2)</u>	26.6 $\pm$ 7.8 (25)

TABLE II: **LIBERO-10 Evaluation.** We evaluate SkipVLA across 3 pick and place tasks in simulation against 2 baselines. We also ablate the indicator heuristics to show the model’s adaptability. We calculate statistics over successful trials only. **Bold:** best; underline: second-best in each backbone block. Numbers shown as Mean $\pm$ Standard Deviation (Median).

data collection or manual annotations.

V. EXPERIMENTS

Our evaluation investigates whether interleaving a fast, point-to-point classical motion planner with a VLA model achieves faster execution and better compute efficiency than purely end-to-end VLA execution without compromising on task success. Specifically, our experiments aim to answer two core questions:

- **Efficiency:** Does offloading free-space transit to a classical planner significantly reduce wall-clock execution time and VLA query frequency across varying task horizons without compromising on task success?
- **Modularity:** Is the framework robust to variations in the underlying planner, latent context representation, and indicator function?

A. Simulation Experiments

On a standardized benchmark, we test for task speedup, VLA query reduction, and task completion.

Task setup & benchmark. We evaluate on 13 top-down pick-and-place tasks drawn from two suites in the LIBERO [29] tabletop benchmark: LIBERO-OBJECT (short-horizon, single-object manipulation) and LIBERO-10 (longer-horizon multi-object manipulation). We deliberately select tasks solvable via top-down grasps since our objective is not to stress-test complex grasp synthesis, but to isolate and measure the efficiency gains of hybrid planning. We train a lightweight multi-head cross-attention scorer with a 3-layer MLP bounding box encoder on the human teleoperation demonstrations from LIBERO using all 50 demonstrations per task, 650 demonstrations in total.

Baselines & evaluation metrics. As our base VLAs, we choose $\pi_{0.5}$ [2], a large-scale pretrained model with 3.5B parameters, and SmolVLA [30], a comparatively smaller and faster model with 0.5B parameters. For both of these VLAs, we use the publicly released fine-tuned weights on LIBERO. We experiment with using our SkipVLA framework on each VLA and report results using different planner algorithms, such as VAMP [6] and cuRoboV2 [7].

For evaluation, each method is repeated across 50 trials per task, measuring success rate (SR, %), total successful

attempts (n_{succ}), wall-clock time (Wall s), and VLA query count (VLA q). We use a hard limit of 600 environment steps in MuJoCo as a timeout.

Results. We present the results in Tab. I and Tab. II. First, we note that, regardless of the planner, SkipVLA matches the base model’s performance. Across both VLA base models, SkipVLA with VAMP as the planner reduces the wall-clock time by 40% and 27% on LIBERO-object and LIBERO-10, respectively. We also see a proportional drop in the number of VLA queries, indicating that the time reduction is largely due to fewer VLA queries. We also note that SkipVLA shows consistent performance regardless of the planner choice. While cuRoboV2 has a slightly higher success rate, VAMP demonstrates a shorter wall-clock time.

Ablation of indicator heuristics. We provide ablations on the heuristics to identify the task boundary. For this ablation, we use changes in object height as an indicator of the transition between reach-and-place and active manipulation; i.e., a change beyond a threshold denotes the transition. We choose Vamp as the planner. From Tab. I and Tab. II, we observe that the height-based heuristic performs comparably to other configurations, showing the stability of SkipVLA. However, our proposed indicator heuristics with the VAMP planner yield the shortest overall wall clock time, indicating the fastest task completion.

B. Real-World Experiments

Experiment setup. To validate our method in the real world, we deployed SkipVLA on a physical 6-DoF YAM manipulator. Our method is tested against two baselines across three tasks in the real world: $\pi_{0.5}$ [2] and MolmoAct2 [31], each fine-tuned on 50 teleoperated demonstrations per task.

We evaluate across three top-down pick-and-place tasks of increasing horizon complexity. In addition to success rate, wall-clock time, and VLA query count, we also measure the total energy consumption per trial on the NVIDIA Jetson Thor onboard compute platform. Each task is evaluated over 20 independent trials:

- **Single-Object:** Put the red block in the box (90-second timeout).
- **Multi-Object:** Put the red and blue blocks in the box (120-second timeout).

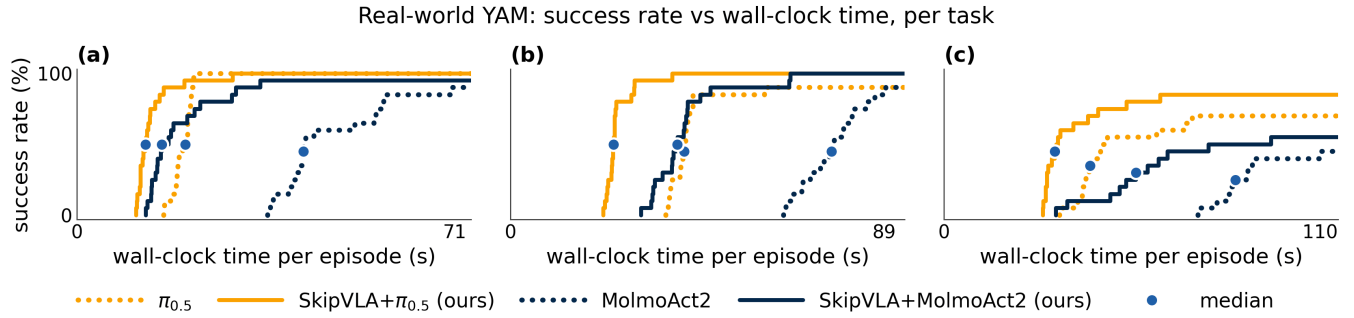


Fig. 4: Cumulative distribution of success rate (%) vs. wall clock (s) for 3 tasks across $\pi_{0.5}$, MolmoAct2, $\pi_{0.5}$ + SkipVLA and MolmoAct2 + SkipVLA measured across 20 trials per task. (a) put the red block in the box. (b) put both the red and blue blocks in the box. (c) stack the 3 blocks in a tower.

Task	Policy	SR $\uparrow$	Wall Time (s) $\downarrow$		VLA Calls $\downarrow$		Energy (J) $\downarrow$	
			mean $\pm$ std	median	mean $\pm$ std	median	mean $\pm$ std	median
Single-Object <i>Put red block in box</i>	$\pi_{0.5}$ (base)	20/20	19.81 $\pm$ 1.73	20.33	9.15 $\pm$ 0.75	9	835.1 $\pm$ 73.9	845.3
	+ SkipVLA (ours)	20/20	14.11 $\pm$ 4.16	12.91	6.10 $\pm$ 1.55	6	577.9 $\pm$ 167.1	539.0
	MolmoAct2 (base)	18/20	46.56 $\pm$ 9.53	42.56	18.06 $\pm$ 3.44	17	1876.6 $\pm$ 383.5	1720.8
	+ SkipVLA (ours)	19/20	18.82 $\pm$ 6.27	15.93	8.16 $\pm$ 2.77	7	712.1 $\pm$ 236.4	610.0
Multi-Object <i>Put red & blue in box</i>	$\pi_{0.5}$ (base)	18/20	41.77 $\pm$ 5.09	41.36	17.89 $\pm$ 2.22	18	1727.0 $\pm$ 214.5	1710.0
	+ SkipVLA (ours)	20/20	25.53 $\pm$ 3.72	24.56	11.15 $\pm$ 1.53	11	1050.8 $\pm$ 146.4	1017.0
	MolmoAct2 (base)	18/20	76.04 $\pm$ 7.17	76.45	29.00 $\pm$ 2.72	29	3236.8 $\pm$ 303.8	3251.4
	+ SkipVLA (ours)	20/20	41.66 $\pm$ 9.48	39.73	17.85 $\pm$ 4.11	17	1562.2 $\pm$ 355.3	1488.1
Precision Stacking <i>Stack tower</i>	$\pi_{0.5}$ (base)	14/20	47.62 $\pm$ 11.91	42.73	20.79 $\pm$ 5.40	19	1976.2 $\pm$ 502.5	1781.2
	+ SkipVLA (ours)	17/20	36.13 $\pm$ 9.68	32.41	16.77 $\pm$ 5.41	15	1631.4 $\pm$ 476.0	1429.9
	MolmoAct2 (base)	9/20	87.13 $\pm$ 9.85	85.26	31.89 $\pm$ 3.59	31	3829.6 $\pm$ 792.3	3621.2
	+ SkipVLA (ours)	11/20	58.11 $\pm$ 17.75	56.19	25.82 $\pm$ 8.12	25	2167.1 $\pm$ 669.2	2112.4

TABLE III: **Real-World Evaluation.** We evaluate SkipVLA against two baselines in the real world on a physical 6-DoF YAM robot arm across three tasks (20 trials per task). Statistics for wall-clock time, VLA calls, and energy are calculated only from successful trials. SR higher is better; all other metrics lower is better; **bold** and shading mark the better value within each backbone block.

- **Multi-Object with precision placing:** Stack three blocks into a tower (120-second timeout).

Main results. Consistent with our simulation results, SkipVLA outperforms both $\pi_{0.5}$ and MolmoAct2 across all tasks and measured metrics, as shown in Tab. III. Specifically, SkipVLA achieves substantial reductions in execution time across all three tasks: wall-clock completion time decreases by an average of 46.0% on MolmoAct2 (up to 59.6%, a $2.47\times$ speedup) and 30.6% on $\pi_{0.5}$. Concurrently, VLA queries drop by an average of 33.8% across tasks (reducing calls by over half on MolmoAct2 for single-object), confirming that offloading free-space transit bypasses a large fraction of costly policy evaluations. Crucially, these efficiency gains come with matched or strictly improved success rates on every task: SkipVLA boosts multi-object placement from 90% to a perfect 100% on both backbones, and raises precision stacking success by +15.0% percentage points for $\pi_{0.5}$ and +10.0% percentage points for MolmoAct2.

Next, in Fig. 4 we report our real-world results as a cumulative distribution that shows the fraction of episodes solved within a given wall-clock budget. As visible from the graphs, SkipVLA encloses a larger area under the curve

than its baseline counterpart on every task, demonstrating that SkipVLA reaches a higher final success rate within a substantially smaller wall-clock budget. By bypassing the base VLA during free-space transit, the hybrid policy drastically reduces the number of required queries to the generative policies and offloads the work to a lightweight, fast, and efficient CPU-based motion planner (VAMP).

Energy comparison. As shown in Tab. III, SkipVLA is also substantially more energy-efficient on the Jetson Thor compute module, cutting per-trial energy consumption by an average of 52.4% on MolmoAct2 and 29.1% on $\pi_{0.5}$. This confirms that our hybrid approach achieves higher physical throughput with a strictly lighter energy footprint.

Additional discussion. Although SkipVLA was designed primarily to reduce inference latency by minimizing VLA queries, we observe that it also improves task success rates, such as improving SmolVLA from 39.0% to 82.0% on LIBERO-OBJECT and improving real-world precision stacking by approximately 20%. We attribute this gain to the planners, which limit compounding errors.

Imitation learning bounds a policy’s error as ϵ when its state remains within the experts’ distribution. During

policy rollout, an out-of-distribution shift in state compounds, incurring a cost bounded by $O(\epsilon T^2)$ over task horizon T [32]. In an end-to-end VLA, a small error during free-space transit can displace the robot out of distribution before it reaches the object, causing grasp failures. SkipVLA mitigates this in two ways: first, restricting VLA queries to short active-manipulation segments of length $T_{\text{manip}} \ll T$ shrinks the time horizon over which errors compound. Second, by planning collision-free paths directly to a nominal hover pose above the object, the classical planner physically re-anchors the robot to in-distribution initial states before each active-manipulation subtask, preventing error propagation to subsequent subtasks.

Lastly, SkipVLA in this work is restricted to pick-and-place tasks due to design choices that were outside the scope of this project: the indicator I depends on gripper-state transitions, which assumes that every active-manipulation segment ends in a grasp or release. Future work aims to make SkipVLA generalizable across all tasks by replacing the heuristic-based indicator with a learned, task-conditioned temporal segmentation module.

VI. CONCLUSION

We present SkipVLA, a method for accelerating pretrained VLAs by offloading free-space motion to a classical motion planner. SkipVLA uses the planner to move the robot to a predicted target pose, hands control to the VLA for contact-rich skills, and returns control to the planner when the gripper opens or closes. A lightweight scoring head on the frozen vision-language backbone of the VLA selects the target object and is trained from existing demonstrations with labels extracted from gripper events. Compared to $\pi_{0.5}$ and MolmoAct2 on a physical 6-DoF YAM arm, SkipVLA completes tasks up to 2.5x faster and uses up to less half the energy of the baseline policy, while matching or improving success rate on every task. The speedup of SkipVLA is larger for the slower MolmoAct2 than for $\pi_{0.5}$ on every task. In simulation, SkipVLA also raises the success rate of SmolVLA significantly on our LIBERO experiments, suggesting that shorter VLA segments limit compounding error. The performance of SkipVLA demonstrates that VLAs can benefit from classical motion planning without retraining the policy.

ACKNOWLEDGMENT

Gemini and Claude assisted with editing and grammar enhancement; all content was verified by the authors.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conf. Robot Learn.*, 2023, pp. 2165–2183.
- [2] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, et al., “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Conf. Robot Learn.*, 2025, pp. 17–40.
- [3] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, et al., “TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 4, pp. 3988–3995, Apr. 2025.
- [4] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Adv. Neural Inf. Process. Syst.*, 2025, pp. 37 596–37 620.
- [5] B. Han, J. Kim, and J. Jang, “A dual process VLA: Efficient robotic manipulation leveraging VLM,” *arXiv:2410.15549*, 2024.
- [6] W. Thomason, Z. Kingston, and L. E. Kavradi, “Motions in microseconds via vectorized sampling-based planning,” in *IEEE Int. Conf. Robot. Autom.*, 2024, pp. 8749–8756.
- [7] B. Sundaralingam, A. Murali, and S. Birchfield, “cuRoboV2: Dynamics-aware motion generation with depth-fused distance fields for high-DoF robots,” *arXiv:2603.05493*, 2026.
- [8] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3D value maps for robotic manipulation with language models,” in *Conf. Robot Learn.*, 2023, pp. 540–562.
- [9] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” in *Conf. Robot Learn.*, 2024, pp. 4573–4602.
- [10] K. Fang, F. Liu, P. Abbeel, and S. Levine, “MOKA: Open-world robotic manipulation through mark-based visual prompting,” in *Robot. Sci. Syst.*, 2024.
- [11] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, et al., “Qwen3-VL technical report,” *arXiv:2511.21631*, 2025.
- [12] Gemma Team, “Gemma 4 technical report,” *arXiv:2607.02770*, 2026.
- [13] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al., “ π_0 : A vision-language-action flow model for general robot control,” in *Robot. Sci. Syst.*, 2025.
- [14] D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, et al., “Octo: An open-source generalist robot policy,” in *Robot. Sci. Syst.*, 2024.
- [15] A. Prasad, K. Lin, J. Wu, L. Zhou, and J. Bohg, “Consistency Policy: Accelerated visuomotor policies via consistency distillation,” in *Robot. Sci. Syst.*, 2024.
- [16] K. Black, A. Z. Ren, M. Equi, and S. Levine, “Training-time action conditioning for efficient real-time chunking,” *arXiv:2512.05964*, 2025.
- [17] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robot. Sci. Syst.*, 2023.
- [18] H. Chen, J. Liu, C. Gu, Z. Liu, R. Zhang, X. Li, X. He, et al., “Fast-in-Slow: A dual-system VLA model unifying fast manipulation within slow reasoning,” in *Adv. Neural Inf. Process. Syst.*, 2025, pp. 108 704–108 738.
- [19] J. Zhang, Y. Guo, X. Chen, Y.-J. Wang, Y. Hu, C. Shi, and J. Chen, “HiRT: Enhancing robotic control with hierarchical robot transformers,” in *Conf. Robot Learn.*, 2024, pp. 933–946.
- [20] Q. Bu, H. Li, L. Chen, J. Cai, J. Zeng, H. Cui, M. Yao, et al., “Towards synergistic, generalized, and efficient dual-system for robotic manipulation,” *arXiv:2410.08001*, 2024.
- [21] T. Chen, H. Wu, J. Wang, X. Li, and L. Fang, “StreamVLA: Breaking the reason-act cycle via completion-state gating,” *arXiv:2602.01100*, 2026.
- [22] T. S. Wilson, W. Thomason, Z. Kingston, and J. D. Gammell, “AORRTC: Almost-surely asymptotically optimal planning with RRT-Connect,” *IEEE Robot. Autom. Lett.*, vol. 10, no. 12, pp. 13 375–13 382, Dec. 2025.
- [23] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” in *IEEE Int. Conf. Robot. Autom.*, 2009, pp. 489–494.
- [24] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, et al., “Motion planning with sequential convex optimization and convex collision checking,” *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, Aug. 2014.
- [25] Y. Yang, S. Cheng, Y. Fang, H. Bharadhwaj, M. Ding, G. Bertasius, and D. Szafir, “LiLo-VLA: Compositional long-horizon manipulation via linked object-centric policies,” *arXiv:2602.21531*, 2026.
- [26] D. B. Choe, S. V. Sangeetha, S. Coogan, and S. Kousik, “Make your VLA more robust without more data by interleaving motion planning,” *arXiv:2606.00985*, 2026.
- [27] X. Zhao, W. Ding, Y. An, Y. Du, T. Yu, M. Li, M. Tang, et al., “Fast segment anything,” *arXiv preprint arXiv:2306.12156*, 2023.
- [28] C. M. Kim, B. Yi, H. Choi, Y. Ma, K. Goldberg, and A. Kanazawa, “Pyroki: A modular toolkit for robot kinematic optimization,” in *IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2025.
- [29] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Adv. Neural Inf. Process. Syst.*, 2023, pp. 44 776–44 791.
- [30] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, et al., “SmolVLA: A vision-language-

- action model for affordable and efficient robotics,” *arXiv:2506.01844*, 2025.
- [31] H. Fang, J. Duan, D. Clay, S. Wang, S. Liu, W. Huang, X. Fan, et al., “MolmoAct2: Action reasoning models for real-world deployment,” *arXiv:2605.02881*, 2026.
- [32] S. Ross and D. Bagnell, “Efficient reductions for imitation learning,” in *Int. Conf. Artif. Intell. Statist.*, 2010, pp. 661–668.